

Static Arrays: Contiguous Memory

Programming and Data Structures — Week 2, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain why indexing into an array costs $O(1)$ in terms of actual memory addresses rather than as a rule memorized without justification; compute, by hand, the address of any element given the array's base address and its element size, in both one and two dimensions; state precisely the one capability a plain, fixed-size array can never provide once it has been created; and explain why a two-dimensional array is, underneath, still a single contiguous one-dimensional block.

Outline

Today covers an apartment-building analogy for contiguous memory; a precise definition of what “contiguous” means physically; the address arithmetic that makes `arr[i]` an $O(1)$ operation, worked through in full with several numeric examples; why storing objects rather than raw numbers does not break this arithmetic; the one hard limitation every fixed-size array runs into; how a two-dimensional array is laid out underneath as one flat block, with the row-major address formula; two complete worked traces; an in-class quiz; and a summary.

The apartment building analogy

Before any code, hold this picture: an array is an apartment building where every flat is exactly the same size and numbered starting at 0. The building manager, asked for flat 47, never walks the corridor counting doors — they compute a location directly: start at the entrance, then move 47 times the width of one flat. This is precisely the mechanism this lecture formalizes. Contrast this with a linked list (covered in Week 4), which behaves like a scavenger hunt: each flat's door only tells you where to find the *next* flat, so reaching flat 47 genuinely requires visiting flats 0 through 46 first. The entire performance difference between arrays and linked lists, which recurs constantly for the rest of this course, traces back to this one distinction.

From one object to many

Week 1 built single objects: one `Point`, one `BankAccount`. Real programs need collections — 1,000 exam scores, not one. `scores = [67, 88, 45, ...]` is the obvious Python answer, but this course cares about what a list actually is underneath, because that determines what it costs to use one.

Contiguous memory: the physical picture

Memory address:	1000	1004	1008	1012	1016
Value:	[67]	[88]	[45]	[92]	[71]
Index:	0	1	2	3	4

An array of 4-byte integers occupies one unbroken block of memory: element 0 at address 1000, element 1 at 1004, element 2 at 1008, and so on. Nothing is scattered and nothing needs to be searched for — the whole point of an array is that the computer knows exactly where every element sits the moment it knows where the array starts. “Contiguous” specifically means there are no gaps between elements — address 1004 belongs entirely to element 1, not partly to something else.

Why 4 bytes, specifically

The element size used in the address formula is not arbitrary — it is fixed by the data type stored, decided once when the array is created and never changed afterward. A standard 32-bit integer requires 4 bytes to represent the full range from -2^{31} to $2^{31} - 1$. A double-precision float requires 8 bytes. This is also *why* an array must declare (or infer) a single element type: mixing a 4-byte integer and an 8-byte float in the same array would break the address formula, since it assumes every element occupies identical space.

The address formula

$$\text{address}(i) = \text{base_address} + i \times \text{element_size}$$

```
base = 1000
element_size = 4      # bytes, for a 4-byte int
i = 3
address = base + i * element_size
print(address)        # 1012 - matches the picture above
```

Computing the address of element `i` takes exactly one multiplication and one addition, no matter whether the array holds 10 elements or 10 million, and no matter whether `i` is 0 or 9,999,999. This is the actual mechanism behind the claim “array indexing is $O(1)$ ” — it is not a rule to memorize, it is a direct consequence of the arithmetic above.

Worked example 1 — small numbers

Take an array of 4-byte integers with base address 500. Applying the formula directly: $\text{address}(0) = 500 + 0 \times 4 = 500$ (the base address itself, since index 0 requires no offset); $\text{address}(1) = 500 + 1 \times 4 = 504$; and $\text{address}(5) = 500 + 5 \times 4 = 520$. Notice the gap between consecutive addresses is always exactly 4, the element size — this constant spacing is what “contiguous” looks like in address arithmetic.

Worked example 2 — a larger element size

Now take an array of 8-byte doubles with base address 2000, and find the address of index 10: $\text{address}(10) = 2000 + 10 \times 8 = 2080$. Comparing this to worked example 1: the same index (imagine index 10 there too) produces a larger offset purely because each element is larger. This matters in practice — an array of doubles occupies twice the memory of an equally-long array of 4-byte integers, and the address arithmetic reflects that directly.

Why this breaks for a list of Point objects

```
points = [Point(0, 0), Point(1, 1), Point(2, 2)]
```

Point objects vary in what they might contain, so Python cannot lay full objects back-to-back the way it can 4-byte integers. Instead, the array stores *references* — fixed-size pointers to where each object actually lives in memory. The array of references is still contiguous and every reference is the same size, so `points[i]` is still $O(1)$: one address calculation to find the reference, then one hop to follow it to the actual Point object.

Tracing the two-hop lookup

Reference array (base 4000, 8-byte pointers):

address 4000: pointer → 6032

address 4008: pointer → 6104

address 4016: pointer → 6176

```
points[1] → address(1) = 4000 + 1*8 = 4008 → read pointer 6104
          → follow to address 6104: the actual Point(1, 1) object
```

This trace makes the “two hops” concrete: `points[1]` first computes address 4008 using the exact same formula as before, reads the 8-byte pointer value stored there (6104), and then follows that pointer to wherever the actual `Point(1, 1)` object lives in memory. Both steps are $O(1)$ — the first is the address formula, the second is a single memory dereference — so the total remains $O(1)$, even though two hops occur instead of one.

The wall: arrays cannot grow

```
scores = [0] * 5      # a fixed block of exactly 5 slots, addresses 1000-1016
scores[5] = 99        # IndexError - there is no address 1020 reserved for us
```

When an array of 5 elements is created, the memory manager reserves exactly those 5 slots and nothing more. The bytes immediately after slot 4 may already be reserved for something else entirely — another variable, another object. Growing the array in place would mean overwriting memory that does not belong to it. The only safe option is to allocate an entirely new, larger block elsewhere and copy everything over — which is exactly the mechanism Lecture 2 covers.

Back to the apartment analogy

Returning to the apartment analogy: a 5-flat building cannot simply grow a 6th flat if the plot next door already belongs to a different owner. The only option is to buy a larger plot elsewhere, move every tenant into the new building, and demolish the old one. This is not a metaphor for dramatic effect — it is literally what a dynamic array does on every resize, as Lecture 2 will trace in full.

Two-dimensional arrays: still one flat block

```
grid = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
```

A two-dimensional array visually suggests a grid of rows and columns, but computer memory is fundamentally one-dimensional — there is no such thing as a “row” at the hardware level, only a single long sequence of addresses. The most common layout, row-major order, stores row 0 completely, then row 1 completely, then row 2, and so on. The 3×3 grid above, flattened this way, is simply the sequence 1, 2, 3, 4, 5, 6, 7, 8, 9 — one contiguous array of 9 elements, with the 2D structure existing only in how addresses are computed from a `(row, col)` pair.

The row-major address formula

$$\text{address}(r, c) = \text{base} + (r \times \text{num_cols} + c) \times \text{element_size}$$

The row-major address formula extends the one-dimensional formula directly: $\text{address}(r, c) = \text{base} + (r \times \text{num_cols} + c) \times \text{element_size}$. The term $r \times \text{num_cols}$ skips past every element belonging to the r complete rows before row r ; adding c then moves across to the correct column within row r itself. This single formula is what every 2D array access compiles down to, in any language.

Worked example 3 — indexing a 2D grid

Take the 3×3 grid from two slides ago, base address 8000, element size 4 bytes, `num_cols = 3`, and find the address of `grid[2][1]`, which holds the value 8. Substituting into the row-major formula: $\text{address}(2, 1) = 8000 + (2 \times 3 + 1) \times 4 = 8000 + 7 \times 4 = 8000 + 28 = 8028$. Two full rows (6 elements) are skipped, then one more element within row 2, giving offset 7 elements, or 28 bytes.

Full worked trace — building a dynamic array’s cousin: a fixed lookup table

A concrete trace end to end: storing 12 months of average temperature as a fixed array, indexed 0 (January) through 11 (December). Finding August’s value is `temps[7]`, since August is the 8th month, index 7. Regardless of which month is requested, the cost is identical — one address computation, one memory read — which is exactly why arrays are the right structure when every element is equally likely to be looked up directly by position.

Live exercise (5 minutes)

An array of 8-byte floats starts at address 2000. Using the formula from this lecture, what address holds `arr[6]`? Separately: for an array declared with 10 elements, what is the last valid index, and why does an off-by-one error so often show up exactly at this boundary? Finally, for a 4×5 grid of 4-byte ints with base address 0 and `num_cols = 5`, what address holds `grid[3][2]`?

Quiz — compute it (1D)

An array of 4-byte integers has base address 3000. Before checking the answer, compute by hand what address holds `arr[7]`.

Quiz — answer (1D)

$\text{address}(7) = 3000 + 7 \times 4 = 3028$. Note that this computation would take exactly the same one multiplication and one addition whether the array held 8 elements or 8 million — the cost of indexing does not depend on array size, which is the entire point of this lecture.

Quiz — compute it (2D)

A 6-row by 4-column grid of 4-byte integers, base address 0, `num_cols = 4`. Before checking the answer, compute by hand what address holds `grid[3][2]`.

Quiz — answer (2D)

$\text{address}(3, 2) = 0 + (3 \times 4 + 2) \times 4 = 0 + 14 \times 4 = 56$. Three complete rows of 4 elements each (12 elements total) are skipped first, then 2 more elements within row 3 are crossed, for an offset of 14 elements, or 56 bytes — matching the same mechanical formula used throughout this lecture.

Summary

An array is one contiguous, fixed-size block of memory, and the address of element `i` is `base + i * element_size` — one multiplication and one addition, regardless of array size, which is precisely why indexing costs $O(1)$. Storing references rather than raw values lets an array hold objects of varying internal size without breaking this formula, at the cost of one extra hop. A two-dimensional array is, underneath, still one flat contiguous block, with row-major order computing $\text{address}(r, c) = \text{base} + (r * \text{num_cols} + c) * \text{element_size}$. A fixed array cannot grow safely in place, because the memory immediately following it is not reserved for it. Next lecture answers the obvious question this raises: Python lists clearly *do* grow — `scores.append(x)` works fine — so what is actually happening underneath when they do, and what does it cost?